

Haskell

Dan Davison

Contents

1	Type	1
2	Function	2
3	Type class	3
4	Functor	3
5	Applicative	4
6	Monoid	5
6.1	Free Monoid Homomorphism theorem	6

1 Type

Definition. A **type** is a set.

Notation.

The type definition syntax $x :: A$ means $x \in A$, i.e. value x has type A .

Example. An example of a type is the set of lists containing items of type a .

Here we declare that variable x is of type list-of-integers and assign a value to x :

```
x :: [Int]
x = [1, 1]
```

And here we define a new type which contains a list of integers, and create a variable of that type:

```
data IntList = MakeIntList [Int]
```

```

y :: IntList
y = MakeIntList [2, 2]

```

We can extract the native list and `Int` values from the `IntList` value by pattern-matching:

```

IntList myList = y
IntList [i, j] = y

```

Now `myList` is `[2, 2]` and `i` and `j` are both 2.

2 Function

Notation.

The type definition syntax `f :: A -> B` means $f : A \rightarrow B$, i.e. f is an element of the set of functions with domain A and codomain B .

Remark. It follows that the type definition syntax `f :: A -> (B -> C)` means f is a function with domain A and codomain the set of functions $B \rightarrow C$. Note that:

1. f can also be thought of as a function with domain $A \times B$ and codomain C .¹
2. By convention the parentheses are omitted and `->` is right-associative. Thus in real Haskell code this would always be written `f :: A -> B -> C`.

Notation. Function application is written without parentheses; parentheses are used for grouping only. Functions of two arguments can be written as infix binary operators. Here are some examples of traditional mathematical notation and Haskell equivalents:

$f(x, y)$		<code>f x y</code>	
$f(g(x))$		<code>f (g x)</code>	<code>f \$ g x</code>
$f(x + y)$	<code>f (+ x y)</code>	<code>f (x + y)</code>	<code>f \$ x + y</code>
$f(x) + f(y)$	<code>+ (f x) (f y)</code>	<code>f x + f y</code>	
$(f \circ g)(x)$		<code>(f . g) x</code>	

The following syntax has no standard equivalent in traditional mathematical notation:

list concatenation	<code>list1 ++ list2</code>	E.g. <code>[1, 2] ++ [3, 4] == [1, 2, 3, 4]</code>
prepend ("cons") to a list	<code>x : list1</code>	E.g. <code>1 : [2, 3, 4] == [1, 2, 3, 4]</code>

¹`A -> B -> C` is known as the "curried" version and `(A x B) -> C` is known as the "uncurried" version.

3 Type class

Definition. A *typeclass* is a set of types.

Remark.

1. A type class is defined using the **class** keyword followed by some methods that all instances of the type class must have. For example

```
class Monoid a where
  mempty  :: a
  mappend :: a -> a -> a
```

2. We can declare that type **A** is an instance of type class **X** in two ways:

- (a) At type-definition time. The class method implementations will be derived automatically. E.g. **data A = MakeA deriving X**
- (b) After type-definition time. Here we can provide our own implementations of class methods. E.g.

```
instance A Monoid
  where mappend x y = x ++ y
```

3. A type signature can have a type class constraint: **f :: X a => a -> B** means: let **a** be any type that is² an instance of type class **X**; then **f :: a -> B**. This is an example of a polymorphic function definition.

4 Functor

Definition.

```
class Functor f where
  (<>) :: (a -> b) -> f a -> f b
```

Remark.

1. Note that the functor **f** is an operator on types. It maps a type to a new type.
2. The new type can be viewed as a “container” or a “context” for objects of type **a**.
3. **(<>)**, also called **fmap**, operates on a function **g :: a -> b**, “lifting” it to operate on objects of type **f a**.

Example.

²In fact, in Haskell one says “has an instance of”, not “is an instance of”.

1. An example of a functor is the list constructor `[]`. In this case `fmap` is just `map`:

```
instance Functor [] where
  (<*>) g (x:xs) = g x : (g <*> xs)
```

2. A second example of a functor is `((->) e)`. It takes a type `a` and maps it to the function type `e -> a`. An example is discussed below under Applicative.

5 Applicative

Definition.

```
class Applicative f where
  (<*>) :: f (a -> b) -> (f a -> f b)
```

Example.

1. An example of a functor is `((->) e)`. It takes a type `a` and maps it to the function type `e -> a`. Let's look into this in more detail.

Suppose we have the predicate functions

```
isAlpha :: Char -> Bool
isDigit :: Char -> Bool
```

and what we want is a nicer version of

```
eitherPredicate :: (Char -> Bool) -> (Char -> Bool) -> (Char -> Bool)
eitherPredicate p1 p2 c = p1 c || p2 c
```

From `ghc/libraries/base/GHC/Base.hs`:

```
instance Functor ((->) r) where
  fmap = (.)

instance Applicative ((->) a) where
  pure = const
  (<*>) f g x = f x (g x)
  liftA2 q f g x = q (f x) (g x)
```

Consider the type of `liftA2`:

```
liftA2 :: Applicative f => (a -> b -> c) -> (f a -> f b -> f c)
```

In the instance where `f` is `((->) e)` this is

```
liftA2 :: (a -> b -> c) -> ((e -> a) -> (e -> b) -> e -> c)
```

So in this Applicative instance, what `liftA2` does is lift a binary operator so that it operates on the *outputs* of two functions.

```
liftA2 (||) isAlpha isDigit :: Char -> Bool
```

So `liftA2` has taken `||` and lifted it to operate on `Char -> Bool` functions.

6 Monoid

Definition. A monoid is a set S with an associative binary operation $\oplus$ and an identity element ϵ .

Remark. A group is a monoid in which each element has an inverse.

Definition. A type is a set.

Definition. A list containing items of type a is a sequence $a_1, a_2, \dots$ where $a_i \in a$ for all $i \in \mathbb{N}$.

Example. The set of lists of type `[a]` is a monoid (the binary operation is concatenation and the identity is the empty list).

Definition. $f : S \rightarrow T$ is a homomorphism if $f(x \oplus y) = f(x) \oplus f(y)$ for all $x, y \in S$.³

Remark.

1. Equivalent notation: $(f \circ \oplus)(x, y) = \oplus(f(x), f(y))$.
2. In some sense this is saying that for a homomorphism f , it doesn't matter in what order we perform f and $\oplus$.

Definition (`map`, `foldr`).

`map` takes a function of one argument and a list of input values and returns a list of output values:

$$\text{map}(g, [x, y, z, \dots]) = [g(x), g(y), g(z), \dots]$$

`foldr` takes a function of two arguments, an initial value, and a sequence of input values, and performs a right-associative "reduce" operation:

$$\text{foldr}(g, \alpha, [x, y, z]) = g(x, f(y, f(z, \alpha))).$$

Remark.

1. When α is an identity element for g and the input is a length-2 list, `foldr` is just function application:

$$\text{foldr}(g, \epsilon, [x, y]) = g(x, g(y, \epsilon)) = g(x, y).$$

³Note that $\oplus$ is used here for two potentially distinct operators: one in S and one in T .

2. Here are type definitions and implementations of `map` and `foldr` in Haskell:

```
map :: (a -> b) -> [a] -> [b]
map g (x:xs) = g x : map g xs
map g [] = []

foldr :: (a -> b -> b) -> b -> [a] -> b
foldr g `alpha` (x:xs) = g x (foldr f `alpha` xs)
foldr g `alpha` [] = `alpha`
```

Instances of the `Monoid` typeclass are defined to have an identity element and a closed binary operation:

```
class Monoid a where
  `id` :: a
  `op` :: a -> a -> a
```

In addition a function `mconcat` is defined for `Monoid` instances. This “reduces” a list of elements:

```
mconcat :: Monoid a => [a] -> a
mconcat = foldr `op` `id`
```

6.1 Free Monoid Homomorphism theorem

Theorem.

Let $b = (b, \oplus, \epsilon)$ be a monoid, and let $f : a \rightarrow b$. Then there exists a unique homomorphism $f' : [a] \rightarrow b$.

I.e. let

```
instance Monoid b where
  `id` :: b
  `op` :: b -> b -> b
```

and let $f :: a \rightarrow b$. Then there exists a unique homomorphism $f' :: [a] \rightarrow b$.

Proof.

We claim that the homomorphism is $f' :: ([a], ++, []) \rightarrow (b, \oplus, \epsilon)$.

Suppose

```
f' [] = `id`
f' [x] = b x
```

Is this a homomorphism? We have

```
f' ([x] ++ [y]) = f' [x, y] =
```

□

Theorem. Let S be a monoid with identity ϵ . Let $x, y \in S$ and let $f : S \rightarrow T$ be a homomorphism. Then

$$f(x \oplus y) = \text{foldr}(\oplus, f(\epsilon), \text{map}(f, [x, y])).$$

I.e.

```
f (x `\$\op\$` y) = foldr `\$\op\$` f(`\$\id\$`) (map f [x, y])
`$\\iff$` (f . `\$\op\$`) x y = (foldr `\$\op\$` f(`\$\id\$`) . (map f)) [x, y]
`$\\iff$` (f . `\$\op\$`) x y = (mconcat . (map f)) [x, y]
`$\\iff$` (f . `\$\op\$`) = mconcat . map f -- well, not exactly,
```

Proof. From the definition of `map` and `foldr` we have:

$$\begin{aligned} \text{foldr}(\oplus, f(\epsilon), \text{map}(f, [x, y])) &= \text{foldr}(\oplus, f(\epsilon), [f(x), f(y)]) \\ &= f(x) \oplus (f(y) \oplus f(\epsilon)) \\ &= f(x) \oplus f(y), \end{aligned}$$

which is equal to $f(x \oplus y)$ since f is a homomorphism. □

Remark. What is this saying?

Yorgey (2012) gives the following:

Definition.

```
hom :: Monoid m => (a -> m) -> ([a] -> m)
```

Theorem.

```
hom f = mconcat . map f
```

Are these saying the same thing?

Let's look at the types of the different components:

```
map f :: [a] `\$\to\$` [f a]
mconcat . map f :: Monoid f a => [a] `\$\to\$` f a
```

So `f a` is a monoid, i.e.

```
f :: Monoid m => a `\$\to\$` m
```